

```
until it finds a solution or reaches the `maxMoves` limit.  
Returns True if a solution was found, in which case `movesMade`  
contains the series of moves to solve the puzzle. Returns False  
if `movesRemaining` is less than 0."""  
  
if movesRemaining < 0:  
    # BASE CASE - Ran out of moves.  
    return False  
  
if board == SOLVED_BOARD:  
    # BASE CASE - Solved the puzzle.  
    return True
```

The JavaScript code for `attemptMove()` begins as follows:

JavaScript

```
function attemptMove(board, movesMade, movesRemaining, prevMove) {  
    // A recursive function that attempts all possible moves on `board`  
    // until it finds a solution or reaches the `maxMoves` limit.  
    // Returns true if a solution was found, in which case `movesMade`  
    // contains the series of moves to solve the puzzle. Returns false  
    // if `movesRemaining` is less than 0.  
  
    if (movesRemaining < 0) {  
        // BASE CASE - Ran out of moves.  
        return false;  
    }  
  
    if (JSON.stringify(board) == SOLVED_BOARD) {  
        // BASE CASE - Solved the puzzle.  
        return true;  
    }  
}
```

The `attemptMove()` function has four parameters. The `board` parameter contains a tile puzzle board data structure to solve. The `movesMade` parameter contains a list or array that `attemptMove()` modifies in place, adding the `UP`, `DOWN`, `LEFT`, and `RIGHT` values that the recursive algorithm has made. If `attemptMove()` solves the puzzle, `movesMade` will contain the moves that led to the solution. This list or array is also what the `solutionMoves` variable in the `solve()` function refers to.